

Smart Contract Test Coverage Report

Auction Smart Contract Testing and Coverage Analysis

Objective

The objective of this testing process was to validate the correctness, reliability, and security of the Auction smart contract using Foundry-based unit tests and coverage analysis.

The testing focused on:

- Auction lifecycle validation
 - Bid placement logic
 - Withdrawal mechanisms
 - Seller verification system
 - Fee management
 - Security protections such as reentrancy prevention
 - Administrative controls
-

Testing Framework Used

Tool	Purpose
------	---------

Foundry	Smart contract development and testing framework
---------	--

Forge	Test execution and coverage analysis
-------	--------------------------------------

Solidity	Smart contract programming language
----------	-------------------------------------

Commands Used

Run Unit Tests

```
forge test
```

Generate Coverage Report

```
forge coverage --ir-minimum
```

Generate LCOV Report

```
forge coverage --report lcov --ir-minimum
```

Test Cases Executed

The following test suites were executed successfully.

Auction Contract Tests

Test Case	Purpose
test_AdminAndSellerManagement	Tests seller registration, verification, and revocation
test_AuctionValidationPaths	Tests auction creation validation logic
test_BidBelowHighestReverts	Ensures low bids revert correctly
test_BiddingAndWithdrawFlows	Tests bidding and withdrawal mechanisms
test_EndAuctionRevertsBeforeDeadline	Verifies auction end restrictions
test_ExtensionAndAuctionEndFlows	Tests seller extension and auction finalization
test_FeesAndViews	Tests fee updates, withdrawals, and view functions
test_LosingBiddersWithdrawExactAmount	Ensures exact withdrawal amounts for losing bidders
test_NoBidsAfterEndAuctionAndSellerReceivesFunds	Verifies seller payout after auction completion
test_Security_ReentrancyGuardPreventsDrain	Tests reentrancy protection

Additional Reentrancy Tests

Test Case	Purpose
test_ReentrancyBlocked	Verifies reentrancy attacks are blocked

Coverage Results

Auction.sol Coverage

Metric Coverage

Line Coverage 92.24%

Statement Coverage 84.83%

Function Coverage 100%

Branch Coverage 62.16%

Overall Project Coverage

Metric Coverage

Total Line Coverage 87.41%

Total Statement Coverage 82.72%

Total Function Coverage 89.29%

Total Branch Coverage 60.98%

File	% Lines	% Statements	% Branches	% Funcs
script/Counter.s.sol	0.00% (0/5)	0.00% (0/3)	100.00% (0/0)	0.00% (0/2)
src/Auction.sol	92.24% (107/116)	84.83% (123/145)	62.16% (23/37)	100.00% (18/18)
src/Counter.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
src/ReentrancyAttacker.sol	100.00% (9/9)	100.00% (6/6)	100.00% (2/2)	100.00% (3/3)
test/ReentrancyAttacker.sol	55.56% (5/9)	50.00% (3/6)	0.00% (0/2)	66.67% (2/3)
Total	87.41% (125/143)	82.72% (134/162)	60.98% (25/41)	89.29% (25/28)

Security Validation

The following security properties were validated:

- Reentrancy protection using OpenZeppelin ReentrancyGuard
- Prevention of unauthorized seller verification
- Prevention of invalid bids
- Prevention of premature auction finalization
- Prevention of double withdrawals

- Proper handling of seller permissions
-

Observations

- The Auction smart contract achieved more than 90% line coverage.
 - All critical auction lifecycle functions were tested successfully.
 - Security-sensitive paths such as withdrawals and reentrancy handling were validated.
 - No failing test cases were observed during execution.
-

Conclusion

The Auction smart contract was tested successfully using Foundry.

The contract achieved:

- High line coverage
- Complete function coverage
- Successful validation of security protections
- Stable execution across all implemented test scenarios

The testing results indicate that the smart contract behaves correctly under both normal and adversarial conditions.